

GENASYS

TECHNICAL ASSESSMENT C# DEVELOPER

July 2025

genasystech.com

REST API

OVERVIEW

This assessment evaluates your technical proficiency in C# development and REST API design. You'll build a simplified e-commerce order processing system.

Submission: Public GitHub repository

PROJECT REQUIREMENTS

System Architecture

Build an **Order Processing Service** consisting of three controllers:

1. **Order Controller** - Manages order creation and status
2. **Inventory Controller** - Tracks product availability
3. **Payment Controller** - Processes payment transactions

Core Functionality

Order Controller

- **POST** /api/orders - Create new order
- **GET** /api/orders/{id} - Retrieve order details
- **GET** /api/orders - List orders (with pagination)
- **PUT** /api/orders/{id}/status - Update order status

Inventory Controller

- **GET** /api/inventory/{productId} - Check product availability
- **POST** /api/inventory/{productId}/reserve - Reserve items
- **POST** /api/inventory/{productId}/release - Release reserved items

Payment Controller

- **POST** /api/payments/process - Process payment
- **GET** /api/payments/{transactionId} - Get payment status

TECHNICAL REQUIREMENTS

Mandatory Technologies

- **.NET 6+** (ASP.NET Core Web API)
- **Entity Framework Core(Or other, Ormlite, etc)** with In-Memory database
- **HTTP Client** for inter-service communication
- **JSON** serialization/deserialization

Code Quality Standards

- **Dependency Injection** throughout the application
- **Async/await** patterns for all I/O operations
- **Proper error handling** with meaningful HTTP status codes
- **Input validation** and model binding
- **Logging** using ILogger interface

genasystech.com

GENASYS HOLDINGS (PTY) LTD
13 Electron Road, Techno Park, Stellenbosch

TEL: +27 860 436 797 | **EMAIL:** info@genasystech.com

GENASYS

LONDON | JOHANNESBURG | CAPE TOWN

DATA MODELS**Order**

```
{  
  "id": "string (GUID)",  
  "customerId": "string",  
  "items": [  
    {  
      "productId": "string",  
      "quantity": "number",  
      "unitPrice": "decimal"  
    }  
  ],  
  "totalAmount": "decimal",  
  "status": "pending|confirmed|cancelled|shipped",  
  "createdAt": "datetime",  
  "updatedAt": "datetime"  
}
```

Inventory Item

```
{  
  "productId": "string",  
  "availableQuantity": "number",  
  "reservedQuantity": "number"  
}
```

Payment Transaction

```
{  
  "transactionId": "string (GUID)",  
  "orderId": "string",  
  "amount": "decimal",  
  "status": "pending|completed|failed",  
  "processedAt": "datetime"  
}
```

BUSINESS LOGIC REQUIREMENTS

Order Creation Flow

1. Validate order data and items
2. Check inventory availability for all items
3. Reserve inventory if available
4. Process payment
5. Confirm order if payment successful
6. Release inventory if payment fails

Error Scenarios to Handle

- Insufficient inventory
- Payment processing failures
- Service unavailability
- Invalid input data
- Concurrent order processing

SUBMISSION REQUIREMENTS

Code Requirements

- **Clean, readable code** with meaningful variable names
- **Comments** explaining complex business logic
- **Consistent code formatting**

EVALUATION CRITERIA

Technical Implementation (60%)

- Code quality and organization
- Proper use of C# and .NET features
- Correct implementation of REST principles
- Effective error handling and validation

Problem Solving (40%)

- Business logic implementation
- Edge case handling
- Performance considerations
- Testing strategy

Bonus Points

Advanced Features (Optional)

- **OpenAPI/Swagger** documentation
- **Authentication/Authorization** (JWT tokens)
- **Caching** implementation (in-memory)
- Configuration management (appsettings)
- Environment-specific settings
- Structured logging with correlation IDs
- Metrics/monitoring endpoints
- Security best practices

SUBMISSION INSTRUCTIONS

1. **Create** a public GitHub repository
2. **Commit** your code with clear, descriptive messages
3. **Email** the repository URL to [interviewer-email]

Pre-submission Checklist

- ☐ Service start successfully
- ☐ End-to-end order flow works correctly
- ☐ Error scenarios are handled gracefully

Remember: This assessment simulates real-world development scenarios. We're interested in your thought process, code organization, and approach to solving distributed systems challenges.